Laboratorio 1 — MSG Foundation: del caso de uso al código

Oct 28, 2026 · @Alberto Alfonso Villarraga Baquero

Datos generales

En este laboratorio cada equipo de cuatro recibe un proyecto Java organizado por capas pero incompleto, reparte el trabajo por roles y lo integra en su propio repositorio de GitHub mediante ramas y pull requests. Todo se hace y se verifica dentro de la sesión.

Campo	Valor
Curso	Ingeniería de Software I (603501) — Unillanos
Unidad	Diseño e implementación
Duración	2 horas (una sesión de práctica por grupo de 20)
Modalidad	Equipos de 4, armados en la sala según la ubicación; cada miembro asume un rol
Caso	MSG Foundation (Schach), nombres de clases en inglés
Lenguaje y herramientas	Java 21, Maven, JUnit 5, Git y GitHub, IDE a elección (IntelliJ, VS Code o NetBeans)
Material	`lab1-msg-base.zip`, publicado en el aula virtual al iniciar la sesión
Interfaz gráfica	No en este laboratorio. JavaFX llega en el Laboratorio 2 y se conecta sin tocar el dominio
Prerrequisito conceptual	GRASP (Experto, Creador, Controlador, Bajo acoplamiento, Alta cohesión), visto en clase. Las capas y los repositorios se presentan brevemente al inicio de la sesión y se estudian a fondo en la clase del 4 de noviembre
Prerrequisito técnico	Antes de la sesión, cada estudiante tiene cuenta de GitHub, Git configurado con su nombre y correo, y ya hizo un push desde un computador de la Sala D a un repositorio de prueba propio

NOTA. El proyecto base trae la estructura de paquetes, los puertos, los repositorios en memoria y todas las pruebas. Las clases donde se toman decisiones ya existen con sus firmas, pero sus métodos están sin implementar y algunas decisiones están tomadas a medias. El equipo completa el código, evalúa las decisiones ya tomadas y toma las que quedan abiertas. Nadie escribe un proyecto desde cero.

Resultados de aprendizaje

Al terminar la sesión, cada estudiante podrá:

1. Justificar con GRASP las responsabilidades ya asignadas en un diseño y tomar las decisiones que quedan abiertas.
2. Ubicar cada clase en su capa (dominio, aplicación, infraestructura) y respetar la regla de que el dominio no importa nada de las otras capas.
3. Implementar reglas de negocio en el dominio y demostrarlas con pruebas unitarias.
4. Trabajar en paralelo sobre el mismo código con ramas, pull requests y revisión cruzada, y explicar por qué un diseño con contratos claros lo hace posible.

El caso: MSG Foundation

La Martha Stockton Greengage (MSG) Foundation otorga hipotecas a parejas jóvenes y cubre con su propio dinero la parte de la cuota que la pareja no alcanza a pagar. Cada semana necesita saber cuánto dinero tendrá disponible para nuevas hipotecas. Ese cálculo es el caso de uso de este laboratorio: **Estimar fondos disponibles para la semana**, junto con **Registrar hipoteca**, que lo alimenta.

Simplificación declarada. Para que el cálculo quepa en dos horas, usamos esta versión de las reglas. Es una adaptación didáctica del caso de Schach, no su transcripción.

Reglas de negocio

Código	Regla
RN1	Cada hipoteca tiene un pago semanal de capital e intereses, un impuesto predial anual, un seguro anual y el ingreso semanal bruto de la pareja.
RN2	El costo semanal de impuesto y seguro es (impuesto anual + seguro anual) / 52.
RN3	La cuota semanal total es el pago de capital e intereses más el costo semanal de impuesto y seguro.
RN4	La pareja aporta como máximo el 28 % de su ingreso semanal bruto.
RN5	El subsidio semanal de la fundación es la cuota total menos el máximo que aporta la pareja; si da negativo, es cero.
RN6	Cada inversión de la fundación tiene un rendimiento anual esperado; su rendimiento semanal es ese valor / 52.
RN7	Los gastos operativos anuales de la fundación se distribuyen igual entre 52 semanas.
RN8	La fundación paga el impuesto y el seguro de todas las hipotecas.

Cálculo de fondos de la semana

```
\text{Fondos} = \sum \text{rendimiento semanal de inversiones} + \sum \text{aporte semanal de parejas} - \text{gasto operativo semanal} - \sum \text{impuesto y seguro semanal}
```

El aporte semanal de una pareja es la cuota total menos el subsidio. Todos los montos se manejan con `BigDecimal`, en dólares, redondeados a 2 decimales.

Ejemplo de verificación

Estos datos son los que usan las pruebas del proyecto base. Si el código no da 1160.00, algo está mal.

Hipoteca	Capital + intereses / semana	Impuesto anual	Seguro anual	Ingreso semanal pareja	Impuesto + seguro / semana	Cuota total	Máx. pareja (28 %)	Subsidio	Aporte pareja
M-001	300.00	2600.00	1040.00	1000.00	70.00	370.00	280.00	90.00	280.00
M-002	200.00	1560.00	520.00	1000.00	40.00	240.00	280.00	0.00	240.00

Inversión	Rendimiento anual	Rendimiento semanal
I-01	52000.00	1000.00
I-02	26000.00	500.00

Gasto operativo anual: 39000.00, es decir 750.00 por semana.

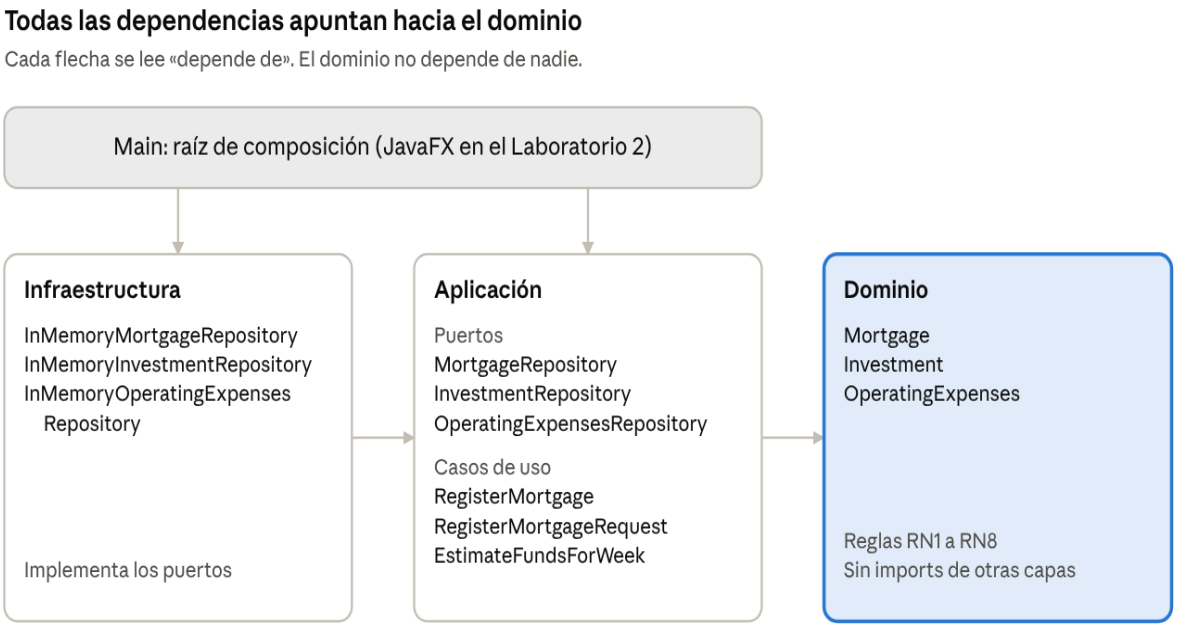
Fondos = 1500.00 + 520.00 − 750.00 − 110.00 = **1160.00**

Antes de empezar: capas, repositorios y GRASP

El código está organizado en tres capas. El **dominio** tiene las reglas del negocio y no sabe que existen una base de datos o una ventana. La **aplicación** tiene los casos de uso, que coordinan el trabajo. La **infraestructura** decide cómo se guardan las cosas. Las capas externas dependen de las internas, nunca al revés.

Repositorio. Es el objeto al que un caso de uso le pide “guárdame esta hipoteca” o “dame todas las hipotecas”. El caso de uso solo conoce la interfaz (`MortgageRepository`), es decir, lo que se puede pedir. No conoce la clase que hace el trabajo (`InMemoryMortgageRepository`), que hoy guarda en memoria y mañana podría guardar en una base de datos. En GRASP, el repositorio es una Fabricación pura, y depender de su interfaz es Bajo acoplamiento. La clase del 4 de noviembre explica por qué este diseño importa tanto.

Ojo con la palabra. En este laboratorio “repositorio” tiene dos significados: el repositorio de GitHub, donde vive el código del equipo, y el repositorio del código Java, que guarda hipotecas e inversiones.



Capas del laboratorio · 3 capas y la raíz de composición

GRASP responde una sola pregunta, de varias formas: ¿a qué clase le asigno esta responsabilidad?

Principio	Pregunta que responde	Respuesta corta
Experto en información	¿Quién calcula o decide esto?	La clase que tiene los datos necesarios
Creador	¿Quién crea un objeto nuevo?	Quien tiene sus datos de inicialización o lo contiene
Controlador	¿Quién recibe una petición del usuario?	Un objeto que representa el caso de uso, no la ventana
Bajo acoplamiento	¿Cómo reduzco el impacto de un cambio?	Depender de pocas clases, y mejor de interfaces
Alta cohesión	¿Esta clase hace demasiado?	Cada clase con un propósito que se explica en una frase
Fabricación pura	¿Y si ninguna clase del dominio encaja?	Inventar una clase técnica, como un repositorio

Roles y flujo de trabajo

En su plan de proyecto para el MSG, Schach reparte el desarrollo así: una persona para las hipotecas, otra para las inversiones y los gastos, y alguien que integra. Este laboratorio sigue ese reparto.

Rol	Rama	Escribe	Revisa el PR de
Integrador	`feature/estimate-funds`	Crea el repositorio; `EstimateFundsForWeek`; fusiona los PR aprobados	Hipotecas
Hipotecas	`feature/mortgage`	`Mortgage`	Registro
Inversiones	`feature/investment`	`Investment` y `OperatingExpenses`	Integrador
Registro	`feature/register-mortgage`	`RegisterMortgage` y `Main`	Inversiones

Si un equipo queda de tres, el Integrador asume también el rol de Registro.

Reglas del flujo

- Nadie hace commit directo en `main`. Todo entra por pull request con una aprobación.
- Cada rama toca solo los archivos de su rol. Si necesita cambiar un archivo ajeno, lo acuerda antes con su dueño.
- Los mensajes de commit dicen qué cambió (“Calcula el subsidio semanal”), no “avance” ni “cambios”.
- Quien revisa un PR deja al menos un comentario, aunque sea para aprobar, y verifica esta lista: las pruebas del rol pasan en la rama; no hay números mágicos; `domain` no importa nada de `application` ni de `infrastructure`; no hay setters ni `double` para dinero.

Contenido del proyecto base

```
lab1-msg-base/
├── pom.xml
├── README.md                 comandos de Git del laboratorio
├── BITACORA.md               plantilla de la entrega
└── src
    ├── main/java/co/edu/unillanos/msg
    │   ├── Main.java                                [por hacer]  Registro
    │   ├── domain
    │   │   ├── Mortgage.java                        [parcial]    Hipotecas
    │   │   ├── Investment.java                      [parcial]    Inversiones
    │   │   └── OperatingExpenses.java               [parcial]    Inversiones
    │   ├── application
    │   │   ├── port                                 [completo]   los tres puertos de
repositorio
    │   │   └── usecase
    │   │       ├── RegisterMortgageRequest.java     [completo]
    │   │       ├── RegisterMortgage.java            [parcial]    Registro
    │   │       └── EstimateFundsForWeek.java        [parcial]    Integrador
    │   └── infrastructure                           [completo]   repositorios en
memoria
    └── test/java/co/edu/unillanos/msg                [completo]   pruebas de todos
los roles
```

Lo que falta lanza `UnsupportedOperationException("TODO")`. Así el proyecto compila desde el primer minuto y `mvn test` arranca en rojo. **Las pruebas no se modifican**; si una parece incorrecta, se discute con el docente.

Así se ve una clase parcial. Las firmas y la descomposición ya están decididas; los cuerpos no:

```
public final class EstimateFundsForWeek {

    // Constructor que recibe los tres puertos: ya escrito

    public BigDecimal execute() {
        return totalInvestmentIncome()
                .add(totalCoupleContributions())
                .subtract(weeklyOperatingExpenses())
                .subtract(totalEscrowPaid());
    }

    private BigDecimal totalInvestmentIncome()    { throw new
UnsupportedOperationException("TODO"); }
    private BigDecimal totalCoupleContributions() { throw new
UnsupportedOperationException("TODO"); }
    private BigDecimal weeklyOperatingExpenses()  { throw new
UnsupportedOperationException("TODO"); }
    private BigDecimal totalEscrowPaid()          { throw new
UnsupportedOperationException("TODO"); }
}
```

Parte 0 — Arranque (15 min)

Mientras el Integrador crea el repositorio, los demás leen las reglas de negocio y la Parte A.

Integrador. Descomprime el .zip, llena en `BITACORA.md` la tabla del equipo con nombres y usuarios de GitHub, y luego:

```
git init -b main
git add .
git commit -m "Código base del Laboratorio 1"
# En GitHub: New repository → lab1-msg-equipoN, público, SIN README
git remote add origin https://github.com/USUARIO/lab1-msg-equipoN.git
git push -u origin main
```

Después, en GitHub:

1. *Settings* → *Collaborators*: invita a los otros tres.
2. *Settings* → *Branches*: crea una regla para `main` con *Require a pull request before merging* y una aprobación. Se hace después del primer push, no antes.

Los demás. Aceptan la invitación en `https://github.com/USUARIO/lab1-msg-equipoN/invitations` (más rápido que esperar el correo) y clonan:

```
git clone https://github.com/USUARIO/lab1-msg-equipoN.git
cd lab1-msg-equipoN
mvn test
```

Resultado esperado: el proyecto compila y las pruebas fallan con `UnsupportedOperationException`. Un `BUILD FAILURE` por pruebas fallidas es normal. Un `COMPILATION ERROR` no lo es: avise al docente.

Parte A — Decisiones del equipo (15 min, sin IA, en papel)

El código base ya tomó algunas decisiones de diseño y dejó otras abiertas. Antes de programar, el equipo justifica las tomadas y decide las abiertas. Cada rol redacta sus filas, pero el equipo entero las discute. La justificación debe nombrar la información o la relación concreta, no solo el principio.

#	Rol	Decisión	Estado	Lo que escribe el equipo
D1	Hipotecas	El subsidio semanal lo calcula `Mortgage`	Tomada	Principio GRASP y qué datos de `Mortgage` lo justifican **Respuesta:** ***Experto en información***, ya que la clase `Mortgage`posee los datos de la cuota y los ingresos de la pareja.
D2	Hipotecas	¿El subsidio y el aporte de la pareja se guardan como atributos al construir la hipoteca, o se calculan cada vez que se piden?	Abierta	Opción elegida y por qué **Respuesta:** Se calcula dinámicamente cada vez que se solicita porque siguiendo los principios de alta cohesión y responsabilidad del dominio, la clase `Mortgage` ya cuenta en su interior con todos los datos para calcular el subsidio y el aporte en tiempo real a través de sus métodos.
D3	Inversiones	`Investment` está declarada `final`	Tomada, revisable	¿La mantienen así? Una ventaja y un riesgo **Respuesta:** Por ahora sí. Una ventaja es que garantiza la seguridad y la inmutabilidad del diseño. Un riesgo es que limita la flexibilidad y reutilización de código. Si en un futuro se desea modelar tipos específicos de inversiones no será posible extender de la clase `Investment`.
D4	Registro	¿Quién detecta un número de hipoteca repetido: `RegisterMortgage` o el repositorio?	Abierta	Opción elegida, principio y qué pasaría si mañana cambia el repositorio **Respuesta:** `RegisterMortgage` lo debe detectar, el principio Grasp es Creador, ya que el repositorio es el encargado de la persistencia de las hipotecas. Si mañana cambia el repositorio, entonces no seria necesario volver a configurarlo para que detecte números id repetidos.
D5	Integrador	`EstimateFundsForWeek` recibe `MortgageRepository` (una interfaz) y no `InMemoryMortgageRepository`	Tomada	Principio GRASP y qué gana el caso de uso con esa decisión **Respuesta:** Bajo acoplamiento, el caso de uso no se acopla a una implementación tecnológica concreta. Esto permite cumplir con el principio de inversión de dependencias, facilitando la prueba unitaria del caso de uso con mocks o stubs y permitiendo cambiar la persistencia en el futuro sin modificar nada en la capa de aplicación.

#	Rol	Decisión	Estado	Lo que escribe el equipo
D6	Integrador	Las sumas sobre todas las hipotecas e inversiones están en métodos privados de `EstimateFundsForWeek`. ¿Se quedan ahí o pasan a un experto nuevo del dominio?	Tomada, revisable	Opción elegida y por qué **Respuesta:** Las entidades del dominio Mortgage y Investment, son expertas en calcular sus valores individuales que son rendimiento semanal y subsidio individual, pero ninguna entidad individual posee la colección global. La colección global es recuperada por los repositorios en la aplicación. Colocar la lógica de suma global en el caso de uso respeta la separación de capas, evitando que el dominio tenga que conocer los repositorios o gestionar listas globales ajenas a su responsabilidad individual.

Parte B — Desarrollo en ramas (35 min, sin IA)

Cada miembro crea la rama de su rol, completa sus clases hasta que pasen sus pruebas, sube la rama y abre un pull request hacia `main`, asignando como revisor a quien indica la tabla de roles.

```
git switch -c feature/mortgage          # la rama de su rol
mvn test -Dtest=MortgageTest            # solo las pruebas de su rol
git add src
git commit -m "Calcula el subsidio semanal de la hipoteca"
git push -u origin feature/mortgage
# En GitHub: Compare & pull request → revisor según la tabla de roles
```

Rol	Pruebas que debe poner en verde en su rama
Hipotecas	`MortgageTest`
Inversiones	`InvestmentTest`, `OperatingExpensesTest`
Registro	`RegisterMortgageTest`. `Main` se prueba después de la integración: debe imprimir 1160.00
Integrador	`EstimateFundsForWeekTest`, que solo pasa cuando entren las ramas de los demás

El Integrador no espera a nadie: programa contra las firmas que ya están en el código base. Que pueda hacerlo es justamente el valor de fijar los contratos antes de repartir el trabajo.

Reglas de implementación

- Los atributos son `private final` y no hay setters. Una hipoteca nace válida o no nace.
- Las divisiones usan `divide(divisor, 2, RoundingMode.HALF_EVEN)`. Una división sin escala puede lanzar `ArithmeticException`.
- Cada porcentaje o constante se declara una sola vez, con nombre, no como número suelto dentro de un método.
- Ninguna clase del dominio imprime en consola ni lee del teclado.
- El paquete `domain` no importa nada de `application` ni de `infrastructure`.
- `EstimateFundsForWeek` no contiene ninguna fórmula de RN2 a RN7: le pide cada valor a la clase experta.

Parte C — Revisión e integración (15 min)

1. Cada revisor revisa el PR que le corresponde con la lista de las reglas del flujo. Aprueba o pide cambios con un comentario concreto.
2. Si piden cambios, el autor hace commit en la misma rama y el PR se actualiza solo.
3. El Integrador fusiona los PR aprobados. El orden da igual, porque cada rol tocó archivos distintos.
4. Todos actualizan su copia y verifican la integración:

```
git switch main
git pull
mvn test          # esperado: BUILD SUCCESS
```

Registro ejecuta `Main` y verifica que imprime 1160.00.

Entregables y evaluación

Todo se entrega antes de salir del aula; no hay trabajo para la casa. Lo que no se vio construir en clase no se califica, y el historial de GitHub muestra el autor y la hora de cada commit.

Qué se entrega

- El repositorio público con la etiqueta `lab1-entrega`.
- Una carpeta docs en el repositorio. Allí, un PDF con la tabla de la Parte A.
- `BITACORA.md` completo en el repositorio: el equipo con sus usuarios de GitHub.

Problemas frecuentes con Git

Síntoma	Causa probable	Solución
El primer push del Integrador es rechazado	El repositorio se creó con README	`git pull --rebase origin main` y vuelve a hacer push, o recrear el repositorio vacío
El Integrador no puede hacer push a `main`	Creó la protección de rama antes del primer push	Quitar la regla, hacer push y volver a crearla
Un miembro no ve la invitación	El correo tarda	Entrar a `github.com/USUARIO/REPO/invitations`
Alguien programó en `main` sin crear rama	Olvidó `git switch -c`	`git switch -c feature/su-rama` conserva los cambios sin commit; luego commit y push normal
`git pull` pide cómo reconciliar ramas	Configuración por defecto de Git reciente	`git config --global pull.rebase false`
Push pide usuario y contraseña	Git Credential Manager no instalado	GitHub no acepta contraseña: usar un token personal o pedir al auxiliar que instale el gestor
Credenciales de un estudiante quedan en el equipo	Equipos compartidos	Al salir: Administrador de credenciales de Windows → borrar las entradas de `git:https://github.com`

Especificación del proyecto base

Archivo	Viene hecho	Queda con `TODO`
`Mortgage`	Clase `final`, constante `WEEKS_PER_YEAR`, los cinco atributos, constructor que asigna con `requireNonNull`, `mortgageNumber()` y `weeklyEscrow()` como ejemplo resuelto	La constante del 28 %, la validación del ingreso en el constructor, `totalWeeklyPayment()`, `maxCoupleContribution()`, `weeklyGrant()` y `weeklyCoupleContribution()`
`Investment`	Clase `final`, atributos `investmentId`, `description` y `annualExpectedReturn`, constructor y getters	`weeklyReturn()`
`OperatingExpenses`	Atributo `annualAmount` y constructor	`weeklyAmount()`
`RegisterMortgage`	Atributo y constructor que recibe `MortgageRepository`; firma `Mortgage execute(RegisterMortgageRequest)`	Cuerpo de `execute()`
`EstimateFundsForWeek`	Constructor con los tres puertos y `execute()` descompuesto en cuatro métodos privados	Los cuatro métodos privados
`Main`	Firma de `main` y un comentario con lo que debe hacer	Armar los repositorios con los datos del ejemplo, ejecutar el caso de uso e imprimir el resultado
Puertos, repositorios en memoria, `RegisterMortgageRequest`	Completos, como en la versión anterior del laboratorio. `InMemoryOperatingExpensesRepository` recibe un `OperatingExpenses` por constructor	—
`README.md`	Los comandos de Git de las Partes 0, B, C y la entrega	—